

Payroll

Status and Way Forward

Where the module stands after the June bank transfer failed to post — what actually caused it, the two fixes now in master, the correction the payment still needs, and the ten issues found alongside it.

DATE 25 August 2026	BUILD master @ 32bfe01e	BLOCKED Payment 16855 — 322,078	OPEN ITEMS 8 of 10
------------------------	----------------------------	---------------------------------------	-----------------------

Contents

1	Summary	5	The workflow question
2	What failed, and why	6	Open issue register
3	What has been refactored	7	Way forward
4	Clearing payment 16855	8	How these figures were verified

1 Summary

June's bank salary transfer — payment **16855, 322,078** — cannot be moved to `PMNT_SENT`. It fails with a message about a missing accrued account on party `276`, which is misleading: party 276 is `موظفي الشركة`, the shared staff party the batch addresses its two settlement payments to. It is not a person and has no salary account by design.

The invoice edit that appeared to trigger this was only the last of **three** changes that moved out from under a payment created seven weeks earlier. The deeper cause is architectural: an aggregate payroll payment records no link to the invoices it settles, so the posting code re-derives that set from live master data every time — and any later edit silently changes the answer.

Two fixes are in master (commit `130429d1`) and need no migration. They stop employee-preference edits from re-partitioning historical runs, and they stop a run payment from being misclassified as an individual one. Together they remove this failure mode for every future run.

They do not unblock payment 16855. The June data is already inconsistent; the code now reports that clearly instead of failing obscurely. The payment still needs the manual correction in section 4.

2 What failed, and why

2.1 Three changes, seven weeks apart

When the batch ran on 5 July it produced two payments, each equal to the sum of the invoices it covered. Both were correct at that moment.

When	What happened	Effect on the June bank run
5 Jul 09:41	Batch created 50 June payroll invoices and both settlement payments. Party 155 (60,000, احمد عبدالرحمن) was BANK_TRANSFER at that moment, so he was part of the bank group.	19 employees 262,078 + party 155 60,000 = 322,078 ✓ matched
19 Jul	Cash payment 16849 sent and posted — 330,684 across 30 employees. Correct, and unaffected by everything below.	—
5 Aug 12:21	Party 155 was edited and his payroll method changed to CASH . This silently removed him from a bank run that had already been executed and paid for.	322,078 → 262,078
25 Aug 07:18	INV1038 (احمد عبد العال, party 169) was reverted to INVOICE_IN_PROCESS , its 2-day absence deduction of 667 removed, and set back to INVOICE_READY .	262,078 → 262,745

By the time the payment was sent, it was being compared against a run total that had moved twice — once by 60,000 and once by 667 — for a net difference of **59,333**.

2.2 Why the error named party 276

The posting code decided *what kind of payment this is* by testing whether its amount still equalled the run total:

```
bool isAggregateRunSettlement =
    string.IsNullOrEmpty(payment.OverrideGlAccountId) && runTotal == paymentAmount;
```

That is a classification derived from arithmetic that is recomputed on every attempt. When the totals stopped agreeing, a whole-run payment was reclassified as a payment to one individual — so the code went looking for an employee salary account on party 276 and, correctly, did not find one.

The message was accurate about the immediate lookup and useless about the cause. Nothing in it mentioned the run, the totals, or the two edits that moved them.

3 What has been refactored

Three changes, all in commit `130429d1`. No database migration: the snapshot uses the existing `INVOICE_ATTRIBUTE` table, which was empty and is already cleaned up by both payroll delete paths.

3.1 Each invoice records the method it was run under

`BatchCreatePayrollInvoices` now writes an `InvoiceAttribute` (`PAYROLL_PMT_METHOD`) on every payroll invoice as it is created, and `GeneralLedgerService` reads that snapshot when deciding which invoices a payment covers — instead of reading the employee's *current* preference.

What it serves: a run becomes a historical fact. Changing an employee's payment method from cash to bank affects the next run and nothing before it. The 5 August edit that removed party 155 from a June run retroactively could not happen again.

The value is stored verbatim rather than normalised to one of the two methods, and the fallback for older invoices keys on the *absence of the attribute row* rather than on a null value. Both details matter: an employee with no preference set belongs to neither payment, and must stay excluded rather than defaulting into one and unbalancing the transaction.

3.2 A payment's identity comes from its recipient, not its arithmetic

The aggregate test is now:

```
bool isAggregateRunSettlement =
    payment.PartyIdTo == PayrollConstants.StaffPartyId
    && string.IsNullOrEmpty(payment.OverrideGlAccountId);
```

The `runTotal == paymentAmount` comparison survives, but demoted from a classification to a *validation* that reports both figures and the difference.

What it serves: a payment can no longer change what it is because a number moved. This is also the marker `GetProjectPayroll`, `GetCompanyReport.GetPayroll` and `Fact_Project_Payroll` already use, so the GL posting and the reports now agree on the definition rather than each inferring it separately.

The same situation now reports itself:

```
Payroll payment 16855 settles the whole BANK_TRANSFER run for 06/2026, but its amount
no longer matches that run: payment 322078.00 vs run total 262745.00 across 19
invoice(s), a difference of 59333.00. The run has changed since the payment was
created (an invoice was edited, or an employee's payroll payment method was changed).
Correct the payment amount or the invoices so the two agree, then post again.
```

3.3 Named constants and two sharper failures

A new `PayrollConstants` class replaces the literals that had to agree across four files but were enforced nowhere: the staff party `276`, the attribute name, and the two method identifiers. Two further failures were upgraded from silent or vague to specific:

- An employee missing an `ACCOUNTS_PAYABLE` account used to log a warning, skip their debit, and then fail the balance guard with bare arithmetic. It now names the employee and invoice.
- The ad-hoc missing-account error now explains what an ad-hoc payroll payment is expected to look like, instead of stating that an account was not found.

Change	Where	What it does <i>not</i> fix
Run-time method snapshot	<code>BatchCreatePayrollInvoices.cs</code> <code>GeneralLedgerService.cs</code>	Existing invoices have no snapshot and still fall back to the live preference
Identity by recipient	<code>GeneralLedgerService.cs</code>	A genuine total mismatch still blocks posting — by design

Change	Where	What it does <i>not</i> fix
Constants and error text	PayrollConstants.cs (new) DeletePayrollInvoices.cs	Nothing behavioural

Do not backfill the snapshot from current preferences. For June that would write `CASH` for party 155 and bake in the exact defect being fixed. The fallback handles historical invoices correctly; set attributes by hand only where the true run membership is known.

4 Clearing payment 16855

The underlying problem is that one payment is doing two jobs: settling nineteen bank employees *and* paying one cash employee. Split it.

Action	Detail
1 Restore the 667 on INV1038	Revert to <code>INVOICE_IN_PROCESS</code> , add the absence item at 2 days (it computes 667 automatically), set back to <code>INVOICE_READY</code> . The invoice returns to 9,333 and re-posts.
2 Set payment 16855 to 262,078	Still <code>PMNT_NOT_PAID</code> , so the amount is editable.
3 Create a second payment for the 60,000	Type <code>PAYROL_PAYMENT</code> , to party 155 , method <code>CIB_CHECKING</code> , effective June. It posts against his own accrued account <code>220008</code> , unpaid since June.
4 Send both	16855 posts as the run settlement (262,078 = 262,078); the 60,000 posts as an individual payment.

	Now	After
INV1038 (احمد عبد العال)	10,000	9,333
Bank run — 19 employees	262,745	262,078
Payment 16855	322,078 ✗	262,078 ✓
Payment to party 155	—	60,000 ✓
Manual journal entry required	667	none

Why restore the deduction rather than journal it

The validation now requires the run payment to equal the run total. Leaving the deduction off the invoice means paying the full 10,000 and clawing 667 back with a journal entry — which leaves the employee's account carrying a debit balance and a correction nobody will recall next year. Putting the deduction back where it belongs makes everything tie out with no journal entry at all.

Side effect of step 1. The payroll item editor saves absence as `quantity 1, amount 667`, whereas the batch saves `quantity 2, amount 333.50`. The money is identical, but the absence report reads `quantity` as days — so that employee will show **1 day** instead of 2. If the day count matters on the report, set the item directly as `quantity 2 / amount 333.50`. This is issue 6 in the register.

5 The workflow question

The user's position is that after running payroll he should be able to correct an employee's invoice and adjust one of the two payments to match. That requirement is legitimate — post-run corrections are normal payroll, and refusing them

outright is not defensible. What is not safe is the mechanism: a hand-typed payment amount alongside a system-derived one gives two sources of truth and no way to tell which is stale. Every part of this incident follows from that.

The line worth drawing is at **send**, not at **run**:

Stage	Rule
Before the payments are sent both are <code>PMNT_NOT_PAID</code>	Edit any invoice freely. The two payment amounts recompute themselves and are never typed by hand. Nothing is posted yet, so this is completely safe — and it is exactly the flexibility being asked for.
After they are sent posted to the GL	The run is closed. Corrections go on next month's invoice or as a separate payment to that employee. The ad-hoc branch already handles this correctly and is the intended escape hatch.

One point deserves to be made without accusation: the payment-method change was almost certainly not understood to matter. Nothing in the UI indicates that editing an employee's payroll method re-partitions runs that have already been executed — because nobody would expect it to. That was a transparency failure on the system's side, and section 3.1 closes it.

Suggested wording for the user. Yes — you can correct an invoice after running payroll. Do it *before* sending the two payments and their amounts will update themselves; you will not type them again. Once the payments are sent the month is closed, and a correction goes on the next run or as a separate payment to that employee. Separately: if someone should be paid by bank instead of cash, change it on the employee *before* running the month — changing it afterwards does not move them between the two payments, it only stops the existing payment from adding up.

6 Open issue register

Found during a full read of the payroll path — backend, GL posting, frontend, seed data — and verified against the local `erp_contracts` database.

#	Issue	Detail	Status
1	No recorded run membership	An aggregate payment records no link to the invoices it settles — no <code>PaymentApplication</code> , no payment group. The set is re-derived on every posting attempt. This is the root of the present incident; 3.1 mitigates it, it does not remove it.	HIGH
2	Two competing deduction engines	<code>BatchCreatePayrollInvoices</code> marks advances deducted, then <code>SetInvoiceStatus(READY)</code> fires <code>ApplyPayrollDeductionsToAdvances</code> , which independently sweeps every approved advance dated in the previous month and marks it fully paid — including advances the run never deducted. Different rules, different status constants, same invoice.	HIGH
3	Invalid schedule status values	The batch writes <code>"PAID"</code> , <code>ApplyPayrollDeductionsToAdvances</code> writes <code>"SCHED_PAID"</code> , reverts write <code>"SCHEDULED"</code> . <code>STATUS_ITEM</code> defines <code>SCHED_PENDING</code> / <code>SCHED_DUE</code> / <code>SCHED_PAID</code> / The table holds 45 <code>SCHEDULED</code> and 26 <code>PAID</code> — neither is a valid status id.	MEDIUM
4	Payroll item types missing from seed	<code>PAYROL_OVERTIME</code> , <code>PAYROL_DD_ABSENCE</code> and <code>PAYROL_DD_ADVANCE</code> exist only in the live database, not in <code>invoice_item_types.json</code> . A fresh environment seeds a payroll module that cannot run.	MEDIUM
5	Attendance computed only in the browser	The fingerprint Excel is parsed entirely client-side. Nothing server-side records the file or how absence days were derived, so a figure cannot be re-audited after the fact.	MEDIUM

#	Issue	Detail	Status
6	Absence days lost on manual edit	The payroll item editor saves <code>quantity 1</code> with the full amount; the batch saves <code>quantity = days</code> with the per-day rate. Money agrees, day counts do not — and both absence and payroll reports read <code>quantity</code> as days.	MEDIUM
7	Hardcoded values	Partly addressed by <code>PayrollConstants</code> . Remaining: <code>CIB_CHECKING</code> , <code>EGP</code> , <code>601000</code> , the 30-day month, and the Friday/Saturday weekend.	LOW
8	<code>CheckExistingPayrollInvoices</code> flaws	Accepts <code>OrganizationPartyId</code> but never applies it, and its month end is <code>monthStart.AddMonths(1)</code> — so the 1st of the next month counts as the current month.	LOW
9	Dead OFBiz heritage	<code>PayrollPreference</code> , <code>SalaryStepNew</code> and <code>PayGrade</code> are mapped but never queried. <code>PayrollReport.tsx</code> is unreferenced; <code>Temps/ListPayrollData.cs</code> is a superseded v1.	LOW
10	Column naming	<code>EMPLOYEE_ADVANCE.AdvanceTypeId</code> is an EF-defaulted <code>longtext</code> , unlike every other snake-case column in the schema.	LOW

7 Way forward

Phase	Work	Why now
Now	Clear payment 16855 per section 4	June's bank salaries are unpaid and one employee's 60,000 has sat in his accrued account since June.
Next	Recompute payment amounts on invoice edit while <code>PMNT_NOT_PAID</code> ; make the amount read-only in the UI; refuse the revert-to- <code>IN_PROCESS</code> path once the payment is sent	This is the change that actually answers the user's request. It grants the flexibility he wants and removes the hand-typed amount that caused the incident. Highest value per unit of effort.
Then	Record run membership explicitly — link each payroll invoice to the payment that settles it	Retires issue 1 and the whole class of failure. Makes an aggregate payment auditable years later instead of reconstructable only by inference.
After	Correctness cleanups: single deduction engine (issue 2), valid status constants with a data fix (3), seed the three item types (4), preserve absence days in the editor (6)	Issue 2 can mark an advance repaid that was never deducted — a real money defect, not tidiness. The rest are cheap once the module is being touched.
Later	Move attendance derivation server-side (issue 5); retire dead code and literals (7, 9, 10)	Auditability rather than correctness. Worth doing before the next audit cycle, not before the next payroll run.

The shape of the fix is consistent throughout: stop deriving historical facts from mutable master data. Snapshot what a run was executed against, record what a payment covers, and let the arithmetic validate rather than classify.

8 How these figures were verified

Every number in this document was read from the local `erp_contracts` database, not inferred from code. The June reconstruction rests on three independent confirmations:

- $262,078 + 60,000 = 322,078$ — the payment's own comment records exactly this composition, written by hand at the time.

- **262,078 + 667 = 262,745** — the current bank run total, after the absence removal.
- **Cash payment 18786 debited 30 employees, not 31** — party 155 is absent from it, confirming he was in the bank group when the run posted, and his June accrual of 60,000 in GL **220008** remains uncleared.

Per-invoice totals were computed the same way the application computes them — signed by **IS_POSITIVE_AMOUNT**, **quantity × amount**, rounded per invoice before summing — which is the rounding rule payroll payments depend on.

```
SELECT p.PREFERRED_PAYROLL_PMT_METHOD method, COUNT(*) n, ROUND(SUM(t.total),2) run_total
FROM INVOICE i JOIN PARTY p ON p.PARTY_ID = i.PARTY_ID_FROM
JOIN (SELECT ii.INVOICE_ID,
        ROUND(SUM(ii.QUANTITY*ii.AMOUNT*CASE WHEN iit.IS_POSITIVE_AMOUNT=0 THEN -1 ELSE 1 END),2) total
      FROM INVOICE_ITEM ii JOIN INVOICE_ITEM_TYPE iit USING(INVOICE_ITEM_TYPE_ID)
      GROUP BY ii.INVOICE_ID) t ON t.INVOICE_ID = i.INVOICE_ID
WHERE i.INVOICE_TYPE_ID='PAYROL_INVOICE' AND i.PARTY_ID='Company'
      AND i.INVOICE_DATE BETWEEN '2026-06-01' AND '2026-06-30'
GROUP BY 1;
```

Prepared 25 August 2026 against master @ 32bfe01e. Sections 3 and 4 reflect code committed in 130429d1; sections 6 and 7 are recommendations, not committed work.